

# 开发日志

---

## 声韵 · 声乐 AI 教学助手 — 开发全日志

---

**生成时间:** 2026-04-28 23:43 ~ 23:55

**产出文件:** `E:\QCLAW\Projects\shengyun-chat\index.html`

### 1. 需求接收

#### 原始需求

接入腾讯元器平台智能体「声韵」（声乐教学 AI 助手）

创建 Web 聊天页面，问题直达智能体并接收回复

智能体信息：      `appid='2049083180719277120'` ,  
                 `appkey='FKUeqc0uvYquY1enhkDxSMZHEh5igpV3'`

API 文档来源：腾讯文档链接 `[https://docs.qq.com/aio/p/sck8w384ntu41e5`](https://docs.qq.com/aio/p/sck8w384ntu41e5)

- 接入腾讯元器平台智能体「声韵」（声乐教学 AI 助手）

#### UI 设计要求

1. 顶部标题栏：🎵 声韵 · 声乐 AI 教学助手 + 副标题"大连艺术学院表演基础教研室出品"
2. 左侧快捷功能区（4 按钮）：气息问题诊断、曲目风格解析、课前热身方案、演唱技巧速查
3. 右侧主对话区：气泡式聊天界面，区分用户/AI 消息
4. 底部输入区：文本输入框 + 发送按钮
5. 风格：声乐/音乐配色（蓝紫调）、舞台戏剧元素、简洁大方、动效 UI、适合教学录屏

1. 顶部标题栏：🎵 声韵 · 声乐 AI 教学助手 + 副标题"大连艺术学院表演基础教研室出品"

### 2. API 文档获取过程

尝试 1: `web_fetch` 直接抓取腾讯文档

**结果:** 失败 X

**原因:** 腾讯文档是 JS 渲染 SPA 页面, Readability 提取器无法获取动态内容

**返回:** 仅有 JS 初始化脚本和配置变量, 无文档正文

- **结果:** 失败 X

尝试 2: browser 工具打开腾讯文档

profile=user → 连接失败 (Chrome MCP attach error)

profile=openclaw → SSRF 策略拦截

**结果:** 失败 X

- profile=user → 连接失败 (Chrome MCP attach error)

尝试 3: web\_fetch text 模式

**结果:** 失败 X

**原因:** 同尝试 1, 返回的是 JS 脚本而非文档内容

- **结果:** 失败 X

尝试 4: 在线搜索 API 文档

使用 online-search Skill (ProSearch)

搜索关键词: `腾讯元器 智能体 API 接口 对话 流式 文档`

**结果:** 成功 ✓

找到腾讯云官方文档:

HTTP SSE 接口 文档 :

`https://cloud.tencent.com/document/product/1759/105561`

WebSocket 接口 文档 :

`https://cloud.tencent.com/document/product/1759/105560`

对话 接口 总体 概述 :

`https://cloud.tencent.com/document/product/1759/109380`

- 使用 online-search Skill (ProSearch)

从搜索结果中提取的关键信息

HTTP SSE 接口地址: `https://wss.lke.cloud.tencent.com/v1/qbot/chat/sse`

这是**腾讯云智能体开发平台**的接口 (lke.cloud.tencent.com) ， 不是**腾讯元器平台**接口

需要区分： 腾讯元器 (yuanqi.tencent.com) vs 腾讯云智能体开发平台 (lke.cloud.tencent.com)

- HTTP SSE 接口地址: `https://wss.lke.cloud.tencent.com/v1/qbot/chat/sse`

3. 用户补充 API 文档

用户在第二轮对话中直接粘贴了完整的 API 文档内容， 关键信息如下：

API 接口信息

| 项目   | 值                                                              |
|------|----------------------------------------------------------------|
| URL  | `https://yuanqi.tencent.com/openapi/v1/agent/chat/completions` |
| 请求方式 | POST                                                           |
| 鉴权方式 | Bearer Token (appkey 即为 token)                                 |

请求参数

| 参数名                 | 类型     | 必选 | 说明                        |
|---------------------|--------|----|---------------------------|
| assistant_id        | string | 是  | 助手 ID (填 appid)           |
| user_id             | string | 是  | 用户 ID                     |
| stream              | bool   | 否  | 是否流式返回， 默认 false          |
| messages            | list   | 是  | 会话内容， 最多 40 条， 按时间从旧到新排列  |
| messages[n].role    | string | 是  | 'user'或'assistant', 交替排列  |
| messages[n].content | list   | 是  | 内容列表， 支持 text 和 image_url |
| custom_variables    | map    | 否  | 自定义参数 (工作流/知识库范围)         |

流式返回格式

SSE 格式: `data: {json}\n\n`

结束标记: `data: [DONE]\n\n`

流式内容在 `choices[0].delta.content` 字段

角色类型: `assistant` (模型输出)、`tool` (工具调用结果)

结束原因：`stop`（正常）、`sensitive`（审核不通过）、`tool\_fail`（工具调用失败）

- SSE 格式：`data: {json}\n\n`

### CURL 示例

```
curl --location 'https://yuanqi.tencent.com/openapi/v1/agent/chat/completions' \
--header 'Content-Type: application/json' \
--header 'Authorization: Bearer appkey' \
--data '{
  "assistant_id": "appid",
  "user_id": "username",
  "stream": true,
  "messages": [
    {
      "role": "user",
      "content": [{"type": "text", "text": "你好"}]
    }
  ]
}'
```

### Python3 示例

```
import requests, json
url = 'https://open.hunyuan.tencent.com/openapi/v1/agent/chat/completions'
headers = {
    'X-Source': 'openapi',
    'Content-Type': 'application/json',
    'Authorization': 'Bearer <appkey>'
}
data = {
    "assistant_id": "I4aVQTHpsJro",
    "user_id": "username",
    "stream": False,
    "messages": [{"role": "user", "content": [{"type": "text", "text": "能创建一个穿着古装的头像吗？"}]}]
}
response = requests.post(url, headers=headers, json=data)
print(response.text)
```

**注意：**Python 示例中的 URL 是 `open.hunyuan.tencent.com`，与 CURL 示例的 `yuanqi.tencent.com` 不同，实际应使用用户文档中的 `yuanqi.tencent.com`。

## 4. 设计决策

### 4.1 技术选型

| 决策点 | 选择        | 理由                 |
|-----|-----------|--------------------|
| 架构  | 单 HTML 文件 | 部署简单，双击即可打开，无需构建工具 |
| CSS | 内联样式      | 无外部依赖，自包含          |

|          |                                             |                     |
|----------|---------------------------------------------|---------------------|
| JS       | 原生 fetch + ReadableStream                   | 流式 SSE 解析，无需第三方库    |
| API 调用方式 | HTTP SSE (stream: true)                     | 用户体验更好，打字机效果        |
| 字体       | Google Fonts (Noto Serif SC + Noto Sans SC) | 衬线标题 + 无衬线正文，中文排版优秀 |

4.2 设计风格

**美学方向:** 舞台/音乐厅 — 深色调戏剧舞台感

| 设计要素 | 实现                                           |
|------|----------------------------------------------|
| 配色   | 深蓝(#0a0a1a)底 + 紫色(#7c5cbf)系 + 金色(#d4a853)点缀  |
| 氛围   | 聚光灯渐变 + 漂浮音符动画 + 紫色光晕漂移                      |
| 字体   | Noto Serif SC (标题, 金色渐变) + Noto Sans SC (正文) |
| 布局   | 左侧 200px 快捷面板 + 右侧 flex 聊天区                  |
| 气泡   | 用户: 蓝紫渐变靠右; AI: 深色半透明靠左 + 紫色边框               |
| 动效   | fadeInUp 消息出现、hover 滑移按钮、脉冲音符、打字机光标          |

4.3 关键功能设计

- 快捷按钮:** 点击直接触发 `sendMessage()`，将预设文本填入输入框并发送
- 对话历史:** 维护 `conversationHistory` 数组，每次请求发送最近 40 条
- 流式解析:** 使用 `ReadableStream` + `TextDecoder` 逐块读取，按 `\\n` 分割解析 SSE 行
- 打字机效果:** 流式追加 `delta.content`，加 `.stream-cursor` CSS 类实现闪烁光标
- 自适应输入框:** 监听 input 事件动态调整 textarea 高度 (max 120px)
- 状态指示:** 在线(绿点) ↔ 思考中(金点脉冲)
- 用户 ID:** `web\_` + 8 位随机字符串，每个页面实例独立
- **快捷按钮:** 点击直接触发 `sendMessage()`，将预设文本填入输入框并发送

5. 开发过程

步骤 1: 读取 frontend-design Skill

路径：`C:\Program Files\QClaw\resources\openclaw\config\skills\frontend-design\SKILL.md`

核心指导：

选定 BOLD 美学方向并精确执行

避免 AI 模板化设计（Inter 字体、紫色渐变白色背景等）

用 CSS 变量保持一致性

动画优先 CSS-only

- 路径：`C:\Program Files\QClaw\resources\openclaw\config\skills\frontend-design\SKILL.md`

步骤 2：创建项目目录

```
mkdir "E:\QCLAW\Projects\shengyun-chat"
```

步骤 3：编写 HTML 文件（第一次）

写入 `E:\QCLAW\Projects\shengyun-chat\index.html`

大小：17,011 bytes

**问题：**write 工具被截断，JS 部分不完整（到 `// State` 注释后就断了）

- 写入 `E:\QCLAW\Projects\shengyun-chat\index.html`

步骤 4：重写完整文件（第二次）

重新写入完整 HTML+CSS+JS

大小：18,658 bytes

完整包含所有功能代码

**成功** ✓

- 重新写入完整 HTML+CSS+JS

步骤 5：浏览器打开测试

```
Start-Process "E:\QCLAW\Projects\shengyun-chat\index.html"
```

页面在默认浏览器中打开

- 页面在默认浏览器中打开

6. 代码结构详解

## HTML 结构

```
body
├── .atmosphere      # 背景紫色光晕动画
├── .spotlight       # 顶部金色聚光灯
├── .music-notes     # 浮动音符容器
└── .app-shell       # 主容器
    ├── .header      # 标题栏
    └── .main-content # 主内容
        ├── .sidebar # 左侧快捷按钮
        │   ├── 4x .quick-btn
        │   └── .sidebar-decor (五线谱装饰)
        └── .chat-area # 聊天区域
            ├── .chat-messages # 消息容器
            ├── .status-bar    # 状态指示
            └── .input-area    # 输入区域
```

## CSS 模块

:root 变量 → Reset → 背景氛围 → 布局 Shell → Header →  
Sidebar → Chat Area → 消息气泡 → 输入区 → 状态指示 →  
Markdown 样式 → 响应式

## JS 模块

CONFIG & State → DOM 引用 → 音符初始化 → textarea 自适应 →  
键盘处理 → quickAsk → 消息渲染(formatContent/addMessage) →  
sendMessage(核心: fetch + ReadableStream SSE 解析)

## SSE 解析逻辑

```
fetch(API_URL, {method:'POST', headers, body})
  → response.body.getReader()
  → reader.read() 循环
  → TextDecoder 解码
  → 按 \n 分割为行
  → 过滤空行和非 data: 开头行
  → JSON.parse 解析
  → 提取 choices[0].delta.content
  → 累加到 fullContent
  → 实时更新 aiBubble.innerHTML
  → data: [DONE] 结束
```

## 7. 已知问题与风险

### ⚠ CORS 跨域（高概率触发）

**问 题**：前 端 直 接 调 用  
`https://yuanqi.tencent.com/openapi/v1/agent/chat/completions`，浏览器  
CORS 策略可能拦截

**表现**：发送消息后控制台报 `Access-Control-Allow-Origin` 错误

## 解决方案：

- 问题：前端直接调用`https://yuanqi.tencent.com/openapi/v1/agent/chat/completions`，浏览器 CORS 策略可能拦截

2. 使用浏览器 CORS 插件（仅开发调试用）

3. 服务器端配置 CORS 头（需腾讯配合，不可行）

2. 使用浏览器 CORS 插件（仅开发调试用）

## API 并发限制

腾讯元器 API 并发限制为 10

当前单用户单会话，不会触发

- 腾讯元器 API 并发限制为 10

## 消息格式

当前仅处理`delta.content`文本内容

未处理`delta.tool\_calls`（工具调用中间步骤）

如智能体调用工具，中间过程的 tool 消息会被跳过，仅显示最终 assistant 回复

- 当前仅处理`delta.content`文本内容

## Python 示例 URL 差异

CURL 示例：

`https://yuanqi.tencent.com/openapi/v1/agent/chat/completions`

Python 示例：

`https://open.hunyuan.tencent.com/openapi/v1/agent/chat/completions`

当前使用：

`https://yuanqi.tencent.com/openapi/v1/agent/chat/completions`（与 CURL 示例一致）

- CURL 示例：

`https://yuanqi.tencent.com/openapi/v1/agent/chat/completions`



8. 文件清单

| 文件         | 路径                                           | 大小      | 说明        |
|------------|----------------------------------------------|---------|-----------|
| index.html | `E:\QCLAW\Projects\shengyun-chat\index.html` | 18,658B | 完整 Web 应用 |

9. 如果需要后续改进

- 1. **CORS 代理**: 添加 `server.js` Node.js 代理层
- 2. **Markdown 渲染**: 引入 marked.js 支持完整 Markdown
- 3. **工具调用展示**: 解析 `tool\_calls` 显示搜索/知识库调用过程
- 4. **会话持久化**: localStorage 保存对话历史
- 5. **多会话**: 支持新建/切换会话
- 6. **移动端适配**: 底部快捷按钮横向滚动替代侧栏

1. **CORS 代理**: 添加 `server.js` Node.js 代理层

10. 本地 AI 工具链 — AI 助手定制训练

- > **定位**: 本项目的本地 AI 内容创作与开发辅助助手，workbuddy 与 QCLAW（技术/代码）构成双轨工具链，协同完成智能体开发。
- > **工具**: workbuddy 与腾讯元器混元大模型 TURBO S
- > **训练时间**: 2026 年 4 月 28 日 — 持续迭代
- > **目标**: 让腾讯元器混元大模型 TURBO S 真正理解声乐教学领域，而非仅作通用 AI 使用。

10.1 训练目标

| 维度   | 目标                              |
|------|---------------------------------|
| 领域知识 | 准确理解呼吸、横膈膜、共鸣腔体、换声点等专业术语        |
| 内容生成 | 自动生成符合中国高等音乐教育规范的教学方案、训练计划、考核建议 |
| 文档撰写 | 辅助撰写申报报告、PPT 内容、口播词等正式材料        |
| 开发辅助 | 参与智能体架构设计、功能模块规划、交互流程优化         |
| 效率提升 | 文档生成效率提升约 70%                   |

10.2 训练方式

## 第一步：知识注入

将声乐教学知识库文档（发声训练、作品风格分析、艺考指导等）作为上下文提供给腾讯元器混元大模型 TURBO S，使其掌握声乐专业知识体系。

### 注入的知识模块：

呼吸与气息训练：腹式呼吸、气息支撑、横膈膜控制

发声与共鸣：喉位、共鸣腔体、音色调控

声区与换声：头声区、混声区、换声点处理

咬字与吐字：十三辙归韵、中文行腔特点

曲目风格与处理：中外艺术歌曲、歌剧咏叹调

艺考备考：曲目选择、考试规范、舞台表现

- 呼吸与气息训练：腹式呼吸、气息支撑、横膈膜控制

**注入方式：**在对话开始时提供知识文档作为背景上下文，要求腾讯元器混元大模型 TURBO S 在后续回复中严格遵循这些内容。

## 第二步：角色设定训练

通过系统提示词（System Prompt）反复调试，确立腾讯元器混元大模型 TURBO S 的"声乐 AI 教学助手"角色定位。

### 角色设定要素：

身份：声乐专业教师，具备中国高等音乐教育背景

风格：语言严谨专业、温和友善、鼓励为主

专业度：引用《声乐艺术论》《嗓音科学》等经典文献时注明出处

边界：超出声乐/音乐领域时，明确提示并给出参考方向

- 身份：声乐专业教师，具备中国高等音乐教育背景

### 调试过程：

1. 初始设定 → 生成测试回答 → 识别偏差（如语气过于机械、内容泛化）
2. 调整提示词 → 再次测试 → 循环迭代
3. 最终固定为角色设定模板，保存至长期记忆

1. 初始设定 → 生成测试回答 → 识别偏差（如语气过于机械、内容泛化）

第三步：多轮对话场景训练

针对智能体的各类典型问题场景进行模拟对话，逐步优化回复质量。

| 场景类型 | 示例问题            | 优化重点               |
|------|-----------------|--------------------|
| 气息问题 | "我唱歌总是气不够怎么办"   | 诊断逻辑清晰、分步骤指导       |
| 换声困难 | "高音总是卡住上不去"     | 技术分析准确、给出具体练习      |
| 曲目选择 | "艺考选什么曲子好"      | 结合学生水平、艺考要求给出个性化建议 |
| 作品解析 | "《我爱你中国》怎么处理情感" | 风格把握、情感层次、演唱处理     |
| 备考咨询 | "声乐艺考要准备哪些内容"   | 全面覆盖、逻辑清晰          |

第四步：反馈迭代

将学生实际使用智能体后的反馈整理后输入 WorkBuddy，持续优化提示词和知识库内容。

反馈处理流程：

1. 收集学生使用反馈（如"气息诊断不够具体"、"曲目风格解析太笼统"）

2. 归类问题类型（知识错误、回复太泛、语气不当）

3. 针对性调整知识库内容或提示词

4. 再次测试，验证改进效果
1. 收集学生使用反馈（如"气息诊断不够具体"、"曲目风格解析太笼统"）

第五步： workflow 协作设计

借助 WorkBuddy 参与智能体的整体开发流程：

| 工作环节  | WorkBuddy 职责                    |
|-------|---------------------------------|
| 架构设计  | 参与智能体功能模块划分、对话流程设计              |
| 文档撰写  | 生成 API 接口说明、开发日志、使用手册、README 文件 |
| 提示词优化 | 撰写并迭代腾讯元器的系统提示词                 |
| 内容审核  | 检查 AI 回复的专业性和准确性                |
| 资源材料  | 整理材料清单、规范文件命名                   |

10.3 训练成效

10.3.1 专业能力提升

- 术语理解：**腾讯元器混元大模型 TURBO S 能够准确理解"横膈膜支撑"、"共鸣腔体"、"换声点"等专业术语，生成的内容无明显专业错误
- 教学方案生成：**可自动生成符合中国高等音乐教育规范的呼吸训练计划、共鸣练习方案、艺考备考规划
- 风格一致性：**回复语言风格统一为"专业、温和、鼓励式"，无通用 AI 的机械感
- **术语理解：**腾讯元器混元大模型 TURBO S 能够准确理解"横膈膜支撑"、"共鸣腔体"、"换声点"等专业术语，生成的内容无明显专业错误

### 10.3.2 效率提升数据

| 工作类型        | 传统方式耗时  | WorkBuddy 辅助耗时 | 效率提升  |
|-------------|---------|----------------|-------|
| 教学方案撰写（单次）  | 约 45 分钟 | 约 12 分钟        | 约 73% |
| 提示词调试（每轮）   | 约 20 分钟 | 约 5 分钟         | 约 75% |
| 申报材料撰写（整份）  | 约 6 小时  | 约 2 小时         | 约 67% |
| 智能体对话测试（每次） | 约 30 分钟 | 约 8 分钟         | 约 73% |

### 10.3.3 典型应用场景

#### \*\*场景一：API 接口说明

任务：说明 API 接口要点

过程：获取腾讯元器接口文档及示例代码 → 获取 QCLAW 任务日志 → 生成说明

结果：生成说明文件

- 任务：说明 API 接口要点

#### 场景二：开发日志生成

任务：生成全效开发日志

过程：提供 workbuddy 本地开发日志 + QCLAW 任务日志 → 集成项目开发日志

结果：生成开发日志文件

- 任务：生成全效开发日志

#### 场景三：提示词工程

任务：优化腾讯元器的系统提示词

- 过程：提供声乐专业知识 + 期望回复风格 → WorkBuddy 生成多版本提示词 → 对比测试选择最优版本
- 任务：优化腾讯元器的系统提示词

#### 10.4 双轨工具链协同总结

| 维度   | WorkBuddy        | QCLAW                              |
|------|------------------|------------------------------------|
| 定位   | 内容/决策            | 技术/代码                              |
| 核心能力 | 专业知识理解、文档生成      | API 逆向、代码编写                        |
| 典型工作 | 提示词撰写、报告生成、知识库建设 | Web 界面开发、Socket.IO 协议解析、Python 客户端 |
| 效率提升 | 文档类工作约 70%       | 技术开发约 65%                          |
| 局限性  | 复杂代码生成能力有限       | 专业领域知识需额外训练                        |

#### 双轨协同模式：

1. **内容→技术**：WorkBuddy 生成功能描述和设计规范 → QCLAW 完成代码实现
  2. **测试→优化**：QCLAW 生成 Web 界面 → WorkBuddy 分析反馈 → QCLAW 根据反馈优化代码
  3. **材料→申报**：WorkBuddy 生成资源材料 → 人工审核 → 最终打包
1. **内容→技术**：WorkBuddy 生成功能描述和设计规范 → QCLAW 完成代码实现

**核心价值**：腾讯元器混元大模型 TURBO S 作为"声乐教学领域的第二大脑"，弥补了通用 AI 在专业领域知识不足的问题，使本地 AI 工具真正成为教学开发的有效助手，而不仅仅是通用问答工具。